LIMKOKWING UNIVERSITY OF CREATIVE TECHNOLOGY
SIERRA LEONE

FACULTY OF INFORMATION AND COMMUNICATION TECHNOLOGY

MODULE ID - MODULE NAME

Title : Individual Assignment
Issue Date : Week 2
Due Date : Week 4
Lecturer/Examiner : Amandus Coker
Name of Student/s: Osman Aamir Kamara
Student ID No. : 905005240
Class : BIT 1102F
Year/Semester : 2/1

Academic Honesty Policy Statement

I/We, hereby attest those contents of this attachment are my own work. Referenced works, articles, art, programs, papers or parts thereof are acknowledged at the end of this paper. This includes data excerpted from CD-ROMs, the Internet, other private networks, and other people's disk of the computer system.

Student's Signature: Date:

LECTURER'S COMMMENTS/GRADE:

for office use only upon receive

Remark

TIME:

DATE

RECEIVER'S NAME:

Mini Library Management System (Python)

1. Data Structures

Books dictionary

```
# ----------------------------
# Mini Library Management System (Python)
# ----------------------------

# === Data Structures ===
books = {}
members = []
genres = ("Fiction", "Non-Fiction", "Sci-Fi", "Biography")

# === Core Functions ===

def add_book(isbn, title, author, genre, total_copies):
    if isbn in books:
        print("❌ Book already exists.")
        return
    if genre not in genres:
        print("❌ Invalid genre. Choose from:", genres)
        return
    books[isbn] = {"title": title, "author": author, "genre": genre,
"total_copies": total_copies}
    print(f"☑ Book '{title}' added successfully.\n")


def add_member(member_id, name, email):
    for m in members:
        if m["member_id"] == member_id:
            print("❌ Member ID already exists.")
            return
    members.append({"member_id": member_id, "name": name, "email": email,
"borrowed_books": []})
    print(f"☑ Member '{name}' added successfully.\n")


def search_books(keyword):
    results = []
    for isbn, info in books.items():
        if keyword.lower() in info["title"].lower() or keyword.lower() in
info["author"].lower():
            results.append((isbn, info))
    if results:
```

```
        print("\n🔎 Search Results:")
        for r in results:
            print(f"ISBN: {r[0]}, Title: {r[1]['title']}, Author:
{r[1]['author']}, Genre: {r[1]['genre']}, Copies: {r[1]['total_copies']}")
    else:
        print("❌ No books found.\n")


def update_book(isbn, title=None, author=None, genre=None, total_copies=None):
    if isbn not in books:
        print("❌ Book not found.")
        return
    if genre and genre not in genres:
        print("❌ Invalid genre.")
        return
    if title:
        books[isbn]["title"] = title
    if author:
        books[isbn]["author"] = author
    if genre:
        books[isbn]["genre"] = genre
    if total_copies is not None:
        books[isbn]["total_copies"] = total_copies
    print("☑ Book updated successfully.\n")


def update_member(member_id, name=None, email=None):
    for m in members:
        if m["member_id"] == member_id:
            if name:
                m["name"] = name
            if email:
                m["email"] = email
            print("☑ Member updated successfully.\n")
            return
    print("❌ Member not found.\n")


def delete_book(isbn):
    if isbn in books:
        for m in members:
            if isbn in m["borrowed_books"]:
                print("❌ Cannot delete: Book is currently borrowed.")
                return
        del books[isbn]
```

```
        print("☑ Book deleted successfully.\n")
    else:
        print("❌ Book not found.\n")


def delete_member(member_id):
    for m in members:
        if m["member_id"] == member_id:
            if m["borrowed_books"]:
                print("❌ Cannot delete: Member has borrowed books.")
                return
            members.remove(m)
            print("☑ Member deleted successfully.\n")
            return
    print("❌ Member not found.\n")


def borrow_book(member_id, isbn):
    for m in members:
        if m["member_id"] == member_id:
            if len(m["borrowed_books"]) >= 3:
                print("❌ Borrow limit reached (max 3 books).")
                return
            if isbn in books and books[isbn]["total_copies"] > 0:
                books[isbn]["total_copies"] -= 1
                m["borrowed_books"].append(isbn)
                print(f"☑ {m['name']} borrowed '{books[isbn]['title']}'.\n")
                return
            else:
                print("❌ Book not available.")
                return
    print("❌ Member not found.\n")


def return_book(member_id, isbn):
    for m in members:
        if m["member_id"] == member_id:
            if isbn in m["borrowed_books"]:
                m["borrowed_books"].remove(isbn)
                books[isbn]["total_copies"] += 1
                print(f"☑ {m['name']} returned '{books[isbn]['title']}'.\n")
                return
            else:
                print("❌ Book not borrowed by this member.")
                return
```

```
    print("❌ Member not found.\n")


# === Simple CLI Menu ===
def menu():
    while True:
        print("""
======== Mini Library System ========
1. Add Book
2. Add Member
3. Search Book
4. Update Book
5. Update Member
6. Delete Book
7. Delete Member
8. Borrow Book
9. Return Book
10. View All Books
11. View All Members
0. Exit
====================================
""")

        choice = input("Enter choice: ")

        if choice == "1":
            isbn = input("ISBN: ")
            title = input("Title: ")
            author = input("Author: ")
            genre = input("Genre: ")
            total = int(input("Total copies: "))
            add_book(isbn, title, author, genre, total)

        elif choice == "2":
            mid = int(input("Member ID: "))
            name = input("Name: ")
            email = input("Email: ")
            add_member(mid, name, email)

        elif choice == "3":
            keyword = input("Enter title or author: ")
            search_books(keyword)

        elif choice == "4":
            isbn = input("ISBN: ")
```

```
            title = input("New Title (blank to skip): ")
            author = input("New Author (blank to skip): ")
            genre = input("New Genre (blank to skip): ")
            copies = input("New Total Copies (blank to skip): ")
            update_book(isbn,
                        title if title else None,
                        author if author else None,
                        genre if genre else None,
                        int(copies) if copies else None)

        elif choice == "5":
            mid = int(input("Member ID: "))
            name = input("New Name (blank to skip): ")
            email = input("New Email (blank to skip): ")
            update_member(mid, name if name else None, email if email else None)

        elif choice == "6":
            isbn = input("ISBN to delete: ")
            delete_book(isbn)

        elif choice == "7":
            mid = int(input("Member ID to delete: "))
            delete_member(mid)

        elif choice == "8":
            mid = int(input("Member ID: "))
            isbn = input("ISBN to borrow: ")
            borrow_book(mid, isbn)

        elif choice == "9":
            mid = int(input("Member ID: "))
            isbn = input("ISBN to return: ")
            return_book(mid, isbn)

        elif choice == "10":
            print("\n📚 All Books:")
            for isbn, b in books.items():
                print(f"{isbn}: {b['title']} ({b['genre']}) - Copies:
{b['total_copies']}")
            print()

        elif choice == "11":
            print("\n👥 All Members:")
            for m in members:
```

```
                print(f"ID: {m['member_id']}, Name: {m['name']}, Borrowed:
{m['borrowed_books']}")
            print()

        elif choice == "0":
            print("👋 Exiting system...")
            break

        else:
            print("❌ Invalid choice. Try again.\n")


# Run the system
if __name__ == "__main__":
    menu()
```

2. Documentation

(a) UML Diagram (structure description)

Library
- Books: Dict - Member: tuple - Genres: tuples
+ add_book() + add_member() + search_books() + update_book() + update_member() + delete_book() + delete_member() | + borrow_book() | | + return_book()

b) Design Rationale

Reason for Using Dictionary, List, and Tuple:

- **Dictionary:** Used for books because it allows quick access and updates using unique keys (ISBNs). Each ISBN maps directly to details like title, author, and available copies.
- **List:** Used for members since we may need to add, iterate, or remove member records flexibly, and each member can hold multiple borrowed books.
- **Tuple:** Used for genres because valid genres are constant, meaning they should not be modified during runtime tuples are immutable and perfect for fixed data.

Advantages:

- Easy data retrieval and updates using keys (dictionary).
- Clear organization of member information (list of dictionaries).
- Data integrity for genres (tuple immutability).
- Simplified CRUD and borrow/return operations using built-in Python structures.

4, GitHub Repo Folder Structure

mini-library-system/

├operations.py # main functions

├demo.py # demo script

├ tests.py # assert-based unit tests

├UML.png # your hand-drawn UML diagram (scan/photo)

├DesignRationale.pdf # 1–2 page explanation

├README.md # how to run the code

operations.py

```
# operations.py
# --------------------------------------------
# Library Management System - Core Functions
# --------------------------------------------

books = {}
members = []
genres = ("Fiction", "Non-Fiction", "Sci-Fi", "Biography")
```

```
def add_book(isbn, title, author, genre, total_copies):
    if isbn in books:
        return "Book already exists"
    if genre not in genres:
        return "Invalid genre"
    books[isbn] = {"title": title, "author": author, "genre": genre,
"total_copies": total_copies}
    return "Book added"

def add_member(member_id, name, email):
    for m in members:
        if m["member_id"] == member_id:
            return "Member exists"
    members.append({"member_id": member_id, "name": name, "email": email,
"borrowed_books": []})
    return "Member added"

def borrow_book(member_id, isbn):
    for m in members:
        if m["member_id"] == member_id:
            if len(m["borrowed_books"]) >= 3:
                return "Limit reached"
            if isbn in books and books[isbn]["total_copies"] > 0:
                books[isbn]["total_copies"] -= 1
                m["borrowed_books"].append(isbn)
                return "Book borrowed"
            else:
                return "Not available"
    return "Member not found"

def return_book(member_id, isbn):
    for m in members:
        if m["member_id"] == member_id:
            if isbn in m["borrowed_books"]:
                m["borrowed_books"].remove(isbn)
                books[isbn]["total_copies"] += 1
                return "Book returned"
            return "Book not borrowed"
    return "Member not found"

def delete_book(isbn):
    if isbn not in books:
        return "Book not found"
    for m in members:
        if isbn in m["borrowed_books"]:
```

```
            return "Book borrowed by a member"
    del books[isbn]
    return "Book deleted"
```

tests.py

```
# tests.py
# ------------------------------------------
# Unit tests using assert statements
# ------------------------------------------
import operations as op

# Reset for testing
op.books.clear()
op.members.clear()

# Test 1: Add a book
assert op.add_book("111", "Python 101", "Muctar Rahim", "Non-Fiction", 3) ==
"Book added"

# Test 2: Add a member
assert op.add_member(1, "Alfred", "Alfred@mail.com") == "Member added"

# Test 3: Borrow a book
assert op.borrow_book(1, "111") == "Book borrowed"
assert op.books["111"]["total_copies"] == 2

# Test 4: Borrow when no copies left
op.books["111"]["total_copies"] = 0
assert op.borrow_book(1, "111") == "Not available"

# Test 5: Return a book
op.books["111"]["total_copies"] = 2
assert op.return_book(1, "111") == "Book returned"

# Test 6: Delete a book
assert op.delete_book("111") == "Book deleted"

print("☑ All tests passed successfully!")
```

UML.png/pdf (hand-draw)